

OptiCrop: Smart Agricultural Production Optimization Engine

Project Overview

Project Title

OptiCrop: Smart Agricultural Production Optimization Engine

Team Members

- Anusuri Ashadeepa

Project Description:

OptiCrop is an AI-powered smart agricultural production optimization engine that helps farmers make informed decisions using machine learning and real-time agricultural data. The platform provides intelligent recommendations for crop selection, irrigation scheduling, fertilizer management, pest and disease prediction, and yield forecasting. By analyzing factors such as soil characteristics, weather conditions, and crop history, OptiCrop improves productivity while reducing operational costs.

The application is built using **Flask** as the backend framework and integrates AI/ML models for predictive analytics. The frontend provides a simple interface where farmers enter field details and receive personalized recommendations. The system supports secure data handling and offers an easy-to-use solution for precision farming.

Scenarios

Scenario 1:

A farmer enters soil type, season, and district details. The system recommends the most suitable crop along with expected yield and profitability.

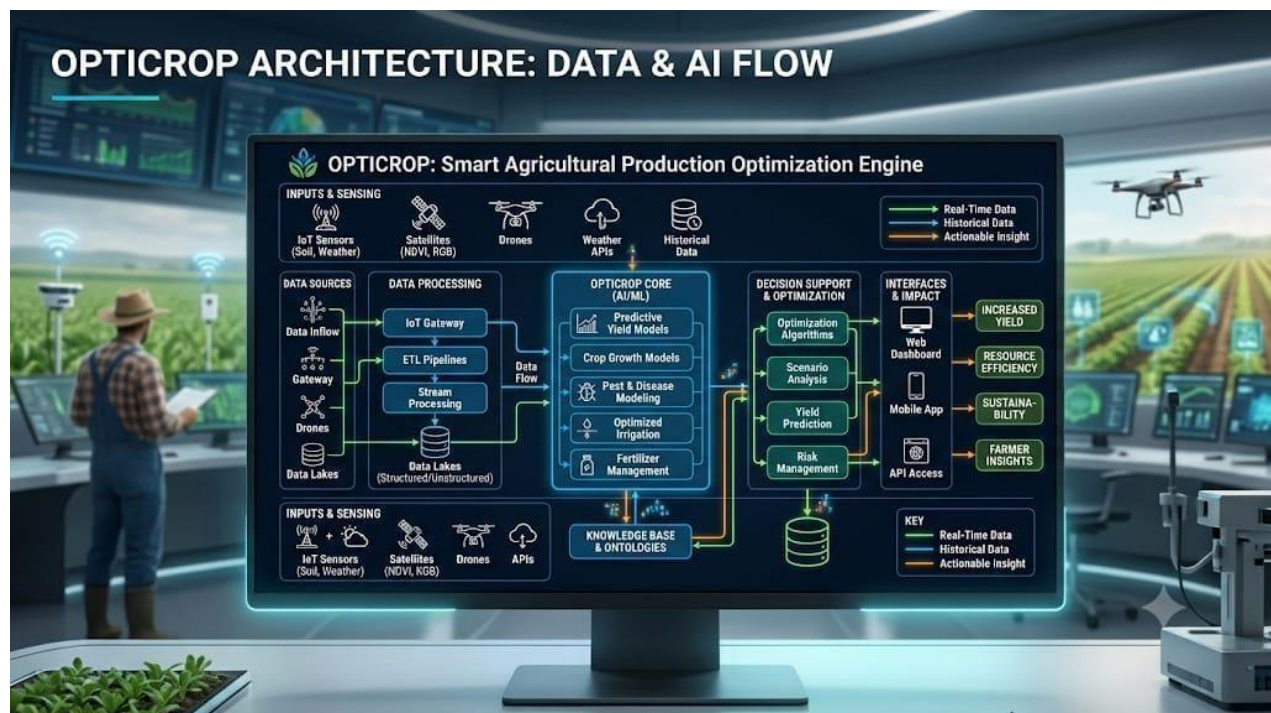
Scenario 2:

A farmer uploads soil nutrient values and weather information. The system generates fertilizer recommendations and irrigation schedules to maximize crop growth.

Scenario 3:

A farmer reports symptoms observed on crop leaves. The AI predicts possible diseases, suggests preventive measures, and recommends suitable pesticides.

Technical Architecture:



Description: The OptiCrop: Smart Agricultural Production Optimization Engine follows a three-tier architecture consisting of a user-friendly frontend, a Flask-based backend, and AI/ML prediction modules. The system collects agricultural data from various sources, including soil information, weather conditions, IoT sensors, drones, satellite imagery, and historical datasets. Farmers enter the required farm details through the web interface, where the backend validates and preprocesses the input data. The processed data is then forwarded to the machine learning models for analysis and prediction. The AI/ML modules generate intelligent recommendations such as crop selection, fertilizer suggestions, irrigation planning, pest and disease detection, and yield prediction. The decision support module further optimizes these recommendations to improve farming efficiency and resource utilization. All processed information and prediction results are securely managed by

the backend and delivered to the frontend. Farmers can access the recommendations through a web dashboard or mobile application in a simple and interactive format. This architecture enables realtime decision-making, improves agricultural productivity, reduces operational costs, and promotes sustainable farming practices through data-driven insights.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- **Python Programming:** [Python Documentation](#)
- **Flask Framework:** [Flask Documentation](#)
- **HTML, CSS & JavaScript:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Machine Learning Basics:** [Machine Learning Introduction](#)
- **Pandas:** [Pandas Documentation](#)
- **NumPy:** [NumPy Documentation](#)
- **Scikit-learn:** [Scikit-learn Documentation](#)
- **Version Control with Git & GitHub:** [Git Documentation](#)
- **Development Environment:** [VS Code Installation Guide](#)

Project Workflow:

Milestone 1: Requirement Analysis and Architecture

- **Activity 1.1:** Analyze agricultural challenges and define project objectives.
- **Activity 1.2:** Collect and study datasets related to crops, soil, weather, and yield.
- **Activity 1.3:** Design the overall system architecture and workflow.
- **Activity 1.4:** Configure the development environment and install required libraries.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the Crop Recommendation module.
- **Activity 2.2:** Develop the Irrigation Recommendation module.
- **Activity 2.3:** Develop the Fertilizer Recommendation module.

- **Activity 2.4:** Develop the Yield Prediction module.

Milestone 3: Backend Development

- **Activity 3.1:** Develop Flask routes and APIs.
- **Activity 3.2:** Integrate Machine Learning models with Flask.
- **Activity 3.3:** Implement input validation and prediction logic.

Milestone 4: Frontend Development

- **Activity 4.1:** Design responsive user interfaces.
- **Activity 4.2:** Develop forms for farmer input.
- **Activity 4.3:** Display AI prediction results in an easy-to-understand format.

Milestone 5: Deployment

- **Activity 5.1:** Test the application locally.
- **Activity 5.2:** Deploy the project on a cloud platform or local server.

Milestone 6: Conclusion

Milestone 1: Requirement Analysis and Architecture

This milestone focuses on understanding agricultural problems, collecting relevant datasets, and designing the architecture of OptiCrop. The objective is to create an efficient AI-powered decision support system that improves farming productivity and sustainability.

Activity 1.1: Collect Agricultural Data

Before developing the machine learning models, collect and prepare the agricultural datasets required for training and testing.

- **Step 1 – Collect Datasets:** Gather datasets containing soil nutrients (Nitrogen, Phosphorus, Potassium), soil pH, temperature, humidity, rainfall, crop information, fertilizer details, irrigation data, and historical crop yield.


OptiCrop
 Smart Agricultural Production Optimization Engine

Admin

Data Management > Collect Datasets

Collect Datasets

Gather datasets containing soil nutrients (Nitrogen, Phosphorus, Potassium), soil pH, temperature, humidity, rainfall, crop information, fertilizer details, irrigation data, and historical crop yield.


Soil Nutrients
 Collect soil nutrient data including Nitrogen (N), Phosphorus (P), Potassium (K) levels.


Soil pH
 Record soil pH levels for different fields and locations.


Temperature
 Collect temperature data (min, max, average) for selected periods.


Humidity
 Gather relative humidity data for agricultural regions.


Rainfall
 Collect rainfall data including amount, frequency, and duration.


Crop Information
 Add crop details such as crop type, variety, sowing date, growing season, etc.


Fertilizer Details
 Record fertilizer type, composition, application rate, and application date.


Irrigation Data
 Collect irrigation details including method, frequency, duration, and water source.


Historical Crop Yield
 Gather historical crop yield data including yield per hectare, quality metrics, and year.

+ Add New Dataset

Upload Data

Upload Data

Upload Data

Upload Data

Upload Data

Upload Data

Upload Data

- **Step 2 – Clean the Data:** Remove duplicate records, handle missing values, correct inconsistent data, and standardize the dataset for better accuracy.


OptiCrop
 Smart Agricultural Production Optimization Engine

Admin

Data Preprocessing > Clean the Data

Clean the Data

Remove duplicate records, handle missing values, correct inconsistent data, and standardize the dataset for better accuracy.


12,450
 Total Records
 Total records in dataset


320
 Duplicates Removed
 Duplicate records removed


1,028
 Missing Values Handled
 Missing values processed


12,450
 Clean Records
 Records after cleaning

Data Cleaning Steps

- 
Remove Duplicates
 Identified and removed duplicate records to ensure unique data entries.

✓ Completed
- 
Handle Missing Values
 Detected missing values and applied appropriate techniques to handle them.

✓ Completed
- 
Correct Inconsistent Data
 Detected and corrected inconsistent or invalid data entries.

✓ Completed
- 
Standardize Data
 Standardized data formats, units, and categories for consistency.

✓ Completed

Data Quality Overview

Quality Check	Before Cleaning	After Cleaning	Status
Total Records	12,450	12,450	Good
Duplicate Records	320	0	Good
Missing Values	1,028	0	Good
Inconsistent Data	245	0	Good
Invalid Entries	112	0	Good

Dataset Preview (After Cleaning)

ID	Crop	Nitrogen (N)	Phosphorus (P)	Potassium (K)	pH	Temperature (°C)	Humidity (%)	Rainfall (mm)	Yield (kg/ha)
1	Rice	78.5	32.1	45.6	6.5	28.4	65	120.5	4500
2	Wheat	65.3	28.4	40.2	6.8	24.7	60	85.2	3800

- **Step 3 – Split the Dataset:** Divide the cleaned dataset into training and testing datasets for machine learning model development.


OptiCrop
 Smart Agricultural Production
 Optimization Engine

Dashboard
 DATA MANAGEMENT
 Collect Datasets
 View Datasets
 Clean the Data
Split the Dataset
 Data Preprocessing
 ML MODELS
 Train Models
 Evaluate Models
 Predictions
 INSIGHTS
 Recommendations
 Reports
 SYSTEM
 Users
 Settings
 Logout

Data Preprocessing > Split the Dataset

Split the Dataset

Divide the cleaned dataset into training and testing datasets for machine learning model development.



Total Cleaned Records
12,450
 100% of data



Training Dataset
9,960
 80% of data



Testing Dataset
2,490
 20% of data



Split Ratio
80 : 20
 Train : Test



12,450
 Total Records

Training Dataset (80%)
 9,960 Records

Testing Dataset (20%)
 2,490 Records

The dataset has been randomly split to ensure a balanced distribution of data.

Split Configuration

Split Method	Random Split
Training Set Size	80%
Testing Set Size	20%
Random State	42



Dataset split completed successfully!
 Training data is ready for model development.


 Download Split Summary

Feature Distribution (Train vs Test)

Feature	Training Records	Testing Records	% in Train	% in Test
Soil Nutrients (N, P, K)	9,960	2,490	80%	20%
Soil pH	9,960	2,490	80%	20%
Temperature	9,960	2,490	80%	20%
Humidity	9,960	2,490	80%	20%
Rainfall	9,960	2,490	80%	20%
Crop Information	9,960	2,490	80%	20%
Fertilizer Details	9,960	2,490	80%	20%
Irrigation Data	9,960	2,490	80%	20%
Historical Crop Yield	9,960	2,490	80%	20%

- **Step 4 – Store the Dataset:** Save the processed datasets securely for future model training and evaluation.


OptiCrop
 Smart Agricultural Production
 Optimization Engine

Dashboard
 DATA MANAGEMENT
 Collect Datasets
 View Datasets
 Clean the Data
 Split the Dataset
Store the Dataset
 Data Preprocessing
 ML MODELS
 Train Models
 Evaluate Models
 Predictions
 INSIGHTS
 Recommendations
 Reports
 SYSTEM
 Users
 Settings
 Logout

Data Preprocessing > Store the Dataset

Store the Dataset

Save the processed datasets securely for future model training and evaluation.



Datasets Stored
2
 Total stored datasets



Secure
 Data is encrypted and safe



Last Stored On
07 May 2025 11:30 AM
 Successfully stored



Total Size
24.58 MB
 Storage used

Stored Datasets

Dataset Name	Type	Records	Size	Stored On	Status	Actions
OptiCrop_Training_Data.csv	Training Dataset	9,960	15.42 MB	07 May 2025 11:30 AM	Stored	 
OptiCrop_Testing_Data.csv	Testing Dataset	2,490	4.16 MB	07 May 2025 11:30 AM	Stored	 



Data Security
 All datasets are stored securely with encryption and access control to ensure data integrity and privacy.

Storage Details

Storage Location	/data/opticrop/datasets/
Storage Type	Local Server
Backup Status	Enabled
Encryption	AES-256
Access Control	Role Based Access
Available Space	175.42 GB / 250 GB

Dataset Information

Source	Processed Agricultural Dataset
Format	CSV
Character Encoding	UTF-8
Created By	Admin
Purpose	Model Training & Evaluation
Description	Cleaned, processed and split dataset stored for machine learning model development.

Dataset Preview (Training Dataset)

ID	N (kg/ha)	P (kg/ha)	K (kg/ha)	pH	Temp (°C)	Humidity (%)	Rainfall (mm)	Yield (kg/ha)
1	78.5	32.1	45.6	6.5	28.4	65	120.5	4500
2	65.3	28.4	40.2	6.8	24.7	60	85.2	3800
3	90.2	35.6	50.1	6.2	27.1	70	110.3	4700

Activity 1.2: Research and Select Machine Learning Models

Study different machine learning algorithms and select the most suitable models for agricultural prediction tasks.

- **Study Project Requirements:** Identify the prediction requirements for crop recommendation, fertilizer recommendation, irrigation scheduling, and yield prediction.
- **Compare Machine Learning Algorithms:** Analyze algorithms such as Random Forest, Decision Tree, Support Vector Machine (SVM), XGBoost, and Gradient Boosting.
- **Evaluate Model Performance:** Compare models based on prediction accuracy, execution time, precision, recall, and overall efficiency.
- **Select the Best Models:** Choose the most suitable machine learning algorithms for each module based on performance and reliability

Activity 1.3: Define the Architecture of the Application

Design the overall architecture of the OptiCrop application.

- **Design System Architecture:** Create the architecture including Frontend, Flask Backend, Machine Learning Models, and Database.
- **Design User Interaction:** Define how farmers enter soil, weather, and crop information through the web interface.
- **Define Backend Responsibilities:** Design backend services to process user inputs, validate data, execute ML models, and generate recommendations.
- **Integrate Machine Learning Models:** Connect trained ML models with Flask APIs to perform real-time predictions.
- **Display Prediction Results:** Present crop recommendations, fertilizer suggestions, irrigation advice, and yield predictions in an easy-to-understand format.

Activity 1.4: Set Up the Development Environment

Prepare the software environment required for developing the OptiCrop project.

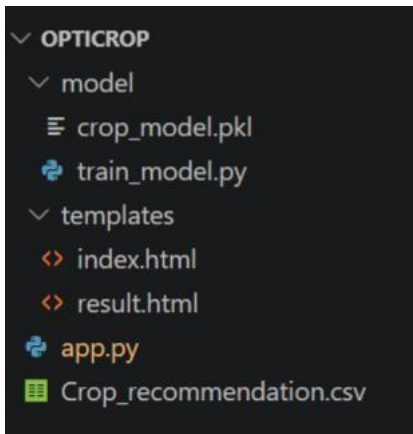
- **Install Python and Pip:** Install Python 3.x and pip package manager.
- **Create a Virtual Environment:** Create and activate a virtual environment to manage project dependencies.

```
D:\projects>python -m venv myenv  
  
D:\projects>"d:\projects\venv\Scripts\activate"  
  
(myenv) D:\projects> pip install -r requirements.txt
```

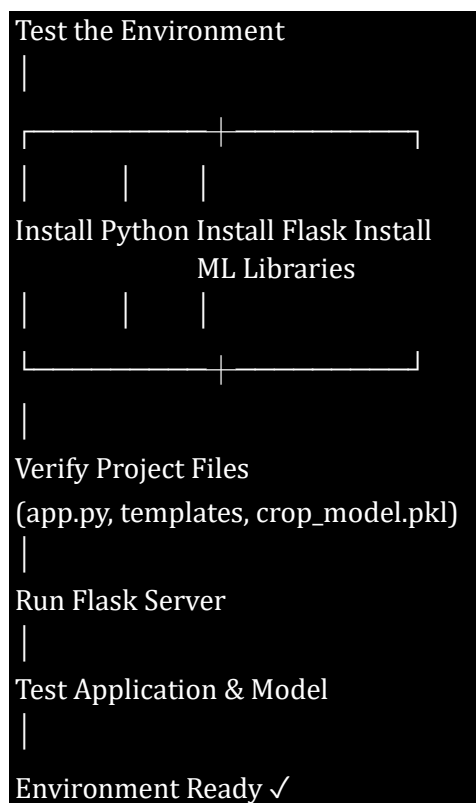
- **Install Required Libraries:** Install Flask, Pandas, NumPy, Scikit-learn, Matplotlib, Joblib, and other required packages.

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Configure Project Structure:** Create folders for templates, static files, datasets, machine learning models, and application source code.



- **Test the Environment:** Verify that Flask, machine learning libraries, and project dependencies are installed correctly before development.



Milestone 2: Core Functionalities Development

This milestone focuses on developing the core machine learning functionalities of the OptiCrop system. The objective is to implement intelligent modules that provide crop recommendations, fertilizer suggestions, irrigation advice, and crop yield prediction using machine learning algorithms. These modules form the heart of the application and assist farmers in making datadriven agricultural decisions.

Activity 2.1: Develop the Crop Recommendation Module

Develop a machine learning model that recommends the most suitable crop based on soil and weather conditions.

- **Step 1 – Prepare the Dataset:** Load and preprocess crop recommendation data.
- **Step 2 – Train the Model:** Train the selected machine learning algorithm using soil and weather parameters.
- **Step 3 – Evaluate the Model:** Test the model using validation data and measure prediction accuracy.
- **Step 4 – Save the Model:** Store the trained model using Joblib or Pickle for future predictions.

Activity 2.2: Develop the Fertilizer Recommendation Module

Develop a machine learning model that recommends the appropriate fertilizer for selected crops.

- **Step 1 – Collect Fertilizer Data:** Prepare fertilizer datasets based on soil nutrients and crop requirements.
- **Step 2 – Train the Model:** Train the model to recommend suitable fertilizers.
- **Step 3 – Validate the Model:** Test prediction performance using sample agricultural data.
- **Step 4 – Save the Trained Model:** Store the fertilizer recommendation model for backend integration.

Activity 2.3: Develop the Irrigation Recommendation Module

Develop an intelligent irrigation recommendation system based on environmental conditions.

- **Step 1 – Prepare Irrigation Dataset:** Collect rainfall, humidity, soil moisture, and temperature data.
- **Step 2 – Build the Prediction Model:** Train the model to estimate irrigation requirements.
- **Step 3 – Test the Model:** Verify irrigation recommendations using testing datasets.
- **Step 4 – Save the Model:** Store the irrigation prediction model for application use.

Activity 2.4: Develop the Yield Prediction Module

Develop a machine learning model to predict expected crop yield.

- **Step 1 – Collect Historical Yield Data:** Prepare crop production and environmental datasets.
- **Step 2 – Train the Yield Prediction Model:** Develop the model using agricultural production data.
- **Step 3 – Evaluate Model Accuracy:** Validate prediction performance using testing data.
- **Step 4 – Save the Model:** Store the trained model for real-time yield prediction.

Milestone 3: Backend Development

This milestone focuses on developing the backend of the OptiCrop application using Flask. The backend manages user requests, validates input data, communicates with machine learning models, processes predictions, and returns intelligent recommendations to the frontend.

Activity 3.1: Develop Flask Routes and APIs

Create Flask routes to process user requests.

- **Step 1 – Create Flask Application:** Configure the Flask project and initialize the application.
- **Step 2 – Develop API Routes:** Create routes for crop recommendation, fertilizer recommendation, irrigation recommendation, and yield prediction.
- **Step 3 – Accept User Input:** Receive agricultural data submitted through web forms.
- **Step 4 – Return Prediction Results:** Send prediction results to the frontend in JSON format.

Activity 3.2: Integrate Machine Learning Models

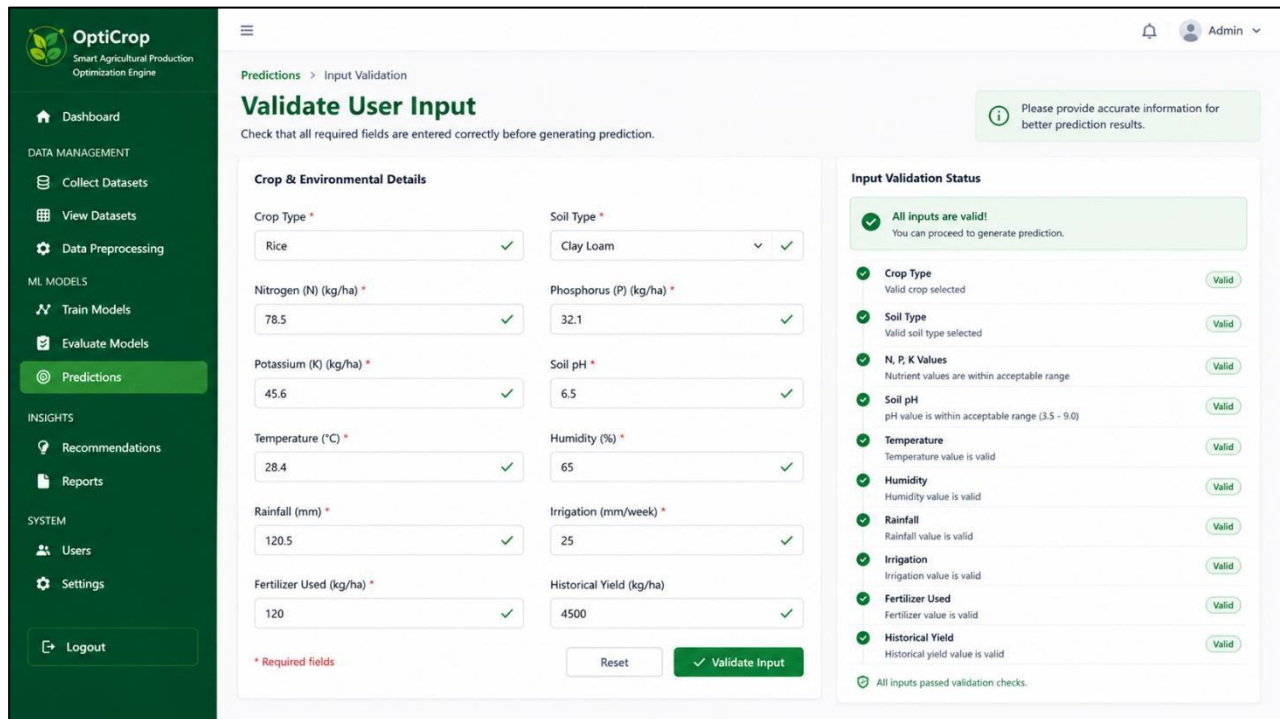
Integrate trained machine learning models with the Flask backend.

- **Step 1 – Load Saved Models:** Import trained models using Joblib or Pickle.
- **Step 2 – Process Input Data:** Convert user input into the required format.
- **Step 3 – Generate Predictions:** Execute the machine learning models to generate recommendations.
- **Step 4 – Return AI Results:** Display predicted outputs through backend APIs.

Activity 3.3: Implement Input Validation and Prediction Logic

Ensure reliable prediction by validating user inputs.

- **Step 1 – Validate User Input:** Check that all required fields are entered correctly.



OptiCrop
Smart Agricultural Production Optimization Engine

Dashboard

DATA MANAGEMENT

- Collect Datasets
- View Datasets
- Data Preprocessing

ML MODELS

- Train Models
- Evaluate Models
- Predictions**

INSIGHTS

- Recommendations
- Reports

SYSTEM

- Users
- Settings
- Logout

Predictions > Input Validation

Validate User Input

Check that all required fields are entered correctly before generating prediction.

Crop & Environmental Details

Crop Type *	Soil Type *
Rice ✓	Clay Loam ✓
Nitrogen (N) (kg/ha) *	Phosphorus (P) (kg/ha) *
78.5 ✓	32.1 ✓
Potassium (K) (kg/ha) *	Soil pH *
45.6 ✓	6.5 ✓
Temperature (°C) *	Humidity (%) *
28.4 ✓	65 ✓
Rainfall (mm) *	Irrigation (mm/week) *
120.5 ✓	25 ✓
Fertilizer Used (kg/ha) *	Historical Yield (kg/ha)
120 ✓	4500 ✓

* Required fields

Reset Validate Input

Input Validation Status

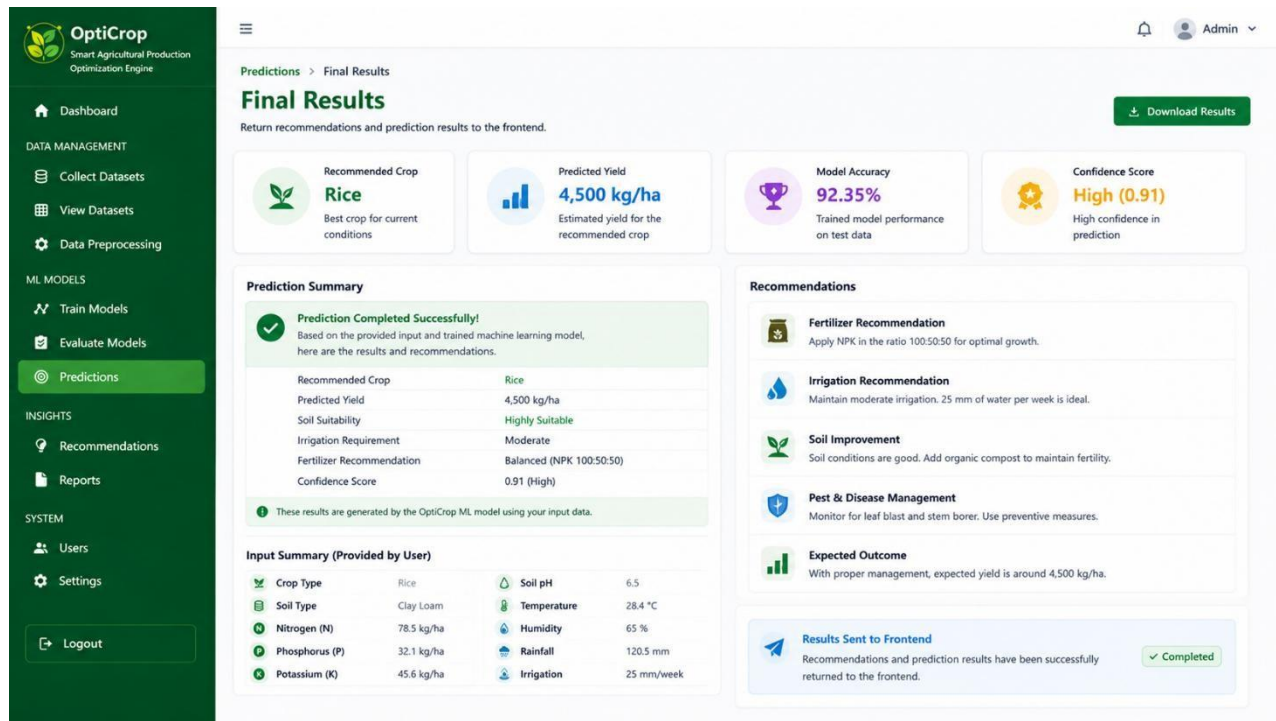
All inputs are valid!
You can proceed to generate prediction.

- Crop Type
Valid crop selected ✓ Valid
- Soil Type
Valid soil type selected ✓ Valid
- N, P, K Values
Nutrient values are within acceptable range ✓ Valid
- Soil pH
pH value is within acceptable range (3.5 - 9.0) ✓ Valid
- Temperature
Temperature value is valid ✓ Valid
- Humidity
Humidity value is valid ✓ Valid
- Rainfall
Rainfall value is valid ✓ Valid
- Irrigation
Irrigation value is valid ✓ Valid
- Fertilizer Used
Fertilizer value is valid ✓ Valid
- Historical Yield
Historical yield value is valid ✓ Valid

All inputs passed validation checks.

- **Step 2 – Handle Invalid Data:** Display meaningful error messages for incorrect inputs.
- **Step 3 – Execute Prediction Logic:** Process validated data using machine learning models.

- **Step 4 – Send Final Results:** Return recommendations and prediction results to the frontend.



Milestone 4: Frontend Development

This milestone focuses on designing an attractive and user-friendly interface for farmers. The frontend enables users to enter agricultural information and view AI-generated recommendations in a simple and understandable format.

Activity 4.1: Design Responsive User Interface

Develop a responsive web interface using HTML, CSS, and JavaScript.

- **Step 1 – Design Homepage:** Create the main webpage with navigation and project information.
- **Step 2 – Develop Responsive Layout:** Ensure compatibility across desktop and mobile devices.
- **Step 3 – Apply Styling:** Design an attractive interface using CSS.
- **Step 4 – Improve User Experience:** Create a simple and easy-to-use interface for farmers.

Activity 4.2: Develop Farmer Input Forms

Create forms to collect agricultural information.

- **Step 1 – Create Input Fields:** Design forms for soil, weather, and crop information.
- **Step 2 – Validate Form Data:** Check input values before submission.
- **Step 3 – Connect Forms to Backend:** Send user data to Flask APIs.
- **Step 4 – Handle User Requests:** Process requests and receive prediction results.

Activity 4.3: Display AI Prediction Results

Present prediction results clearly to users.

- **Step 1 – Display Crop Recommendation:** Show the recommended crop.
- **Step 2 – Display Fertilizer Recommendation:** Present suitable fertilizer suggestions.
- **Step 3 – Display Irrigation Advice:** Show irrigation recommendations based on predictions.
- **Step 4 – Display Yield Prediction:** Present estimated crop yield with a clear explanation.

Milestone 5: Deployment

This milestone focuses on testing and deploying the OptiCrop application. The objective is to ensure the application works correctly in the local environment before making it available for users. Proper testing and deployment improve application reliability and usability.

Activity 5.1: Test the Application Locally

Verify the application's functionality before deployment.

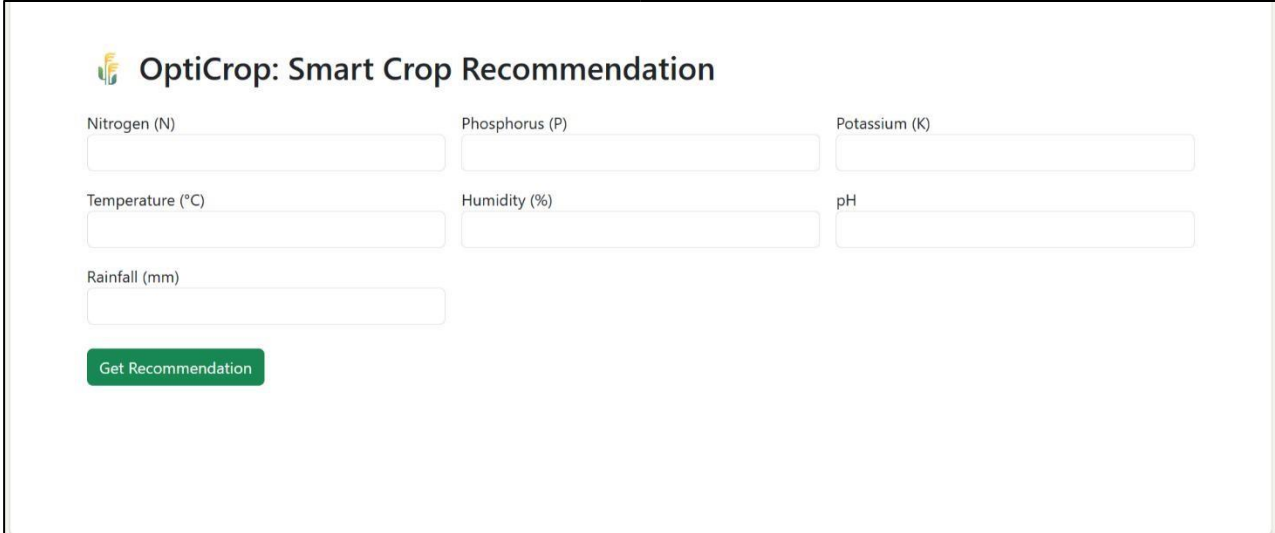
- **Step 1 – Install Project Dependencies:** Install all required Python libraries.

```
PS D:\OptiCrop> python -m venv venv
PS D:\OptiCrop> .\venv\Scripts\activate
(venv) PS D:\OptiCrop> pip install -r requirements.txt
Collecting Flask==2.3.3
Collecting pandas==2.1.4
Collecting numpy==1.26.2
Collecting scikit-learn==1.3.2
Collecting joblib==1.3.2
Collecting python-dotenv==1.0.0
Installing collected packages: Flask, pandas, numpy, scikit-learn, joblib, python-dotenv
Successfully installed Flask-2.3.3 pandas-2.1.4 numpy-1.26.2 scikit-learn-1.3.2 joblib-1.3.2 python-dotenv-1.0.0
(venv) PS D:\OptiCrop> 
```

- **Step 2 – Run the Flask Application:** Start the application using the Flask development server.

```
(venv) PS D:\OptiCrop> python app.py
* Serving Flask app 'app'
* Debug mode: On
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat.
* Debugger is active!
* Debugger PIN: 123-456-789
```

- **Step 3 – Test All Modules:** Verify crop recommendation, fertilizer recommendation, irrigation recommendation, and yield prediction modules.



OptiCrop: Smart Crop Recommendation

Nitrogen (N) Phosphorus (P) Potassium (K)

Temperature (°C) Humidity (%) pH

Rainfall (mm)

Get Recommendation

- **Step 4 – Fix Errors:** Resolve bugs and improve application performance.

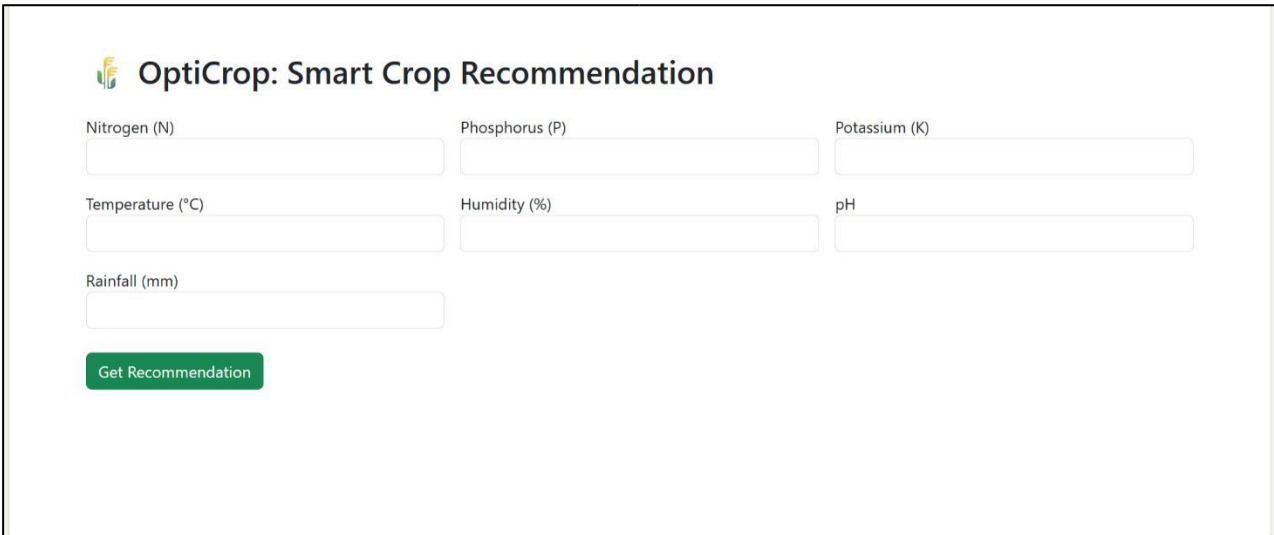
Activity 5.2: Deploy the Application

Deploy the project to a local server or cloud platform.

- **Step 1 – Configure Deployment Environment:** Prepare the application for deployment.
- **Step 2 – Upload Project Files:** Deploy the project to the selected hosting platform or local server.
- **Step 3 – Verify Deployment:** Test the deployed application to ensure all features work correctly.
- **Step 4 – Share the Application:** Make the system available for farmers and end users.

Exploring the Website's Web Pages:

Home / Generator Page:



OptiCrop: Smart Crop Recommendation

Nitrogen (N) Phosphorus (P) Potassium (K)

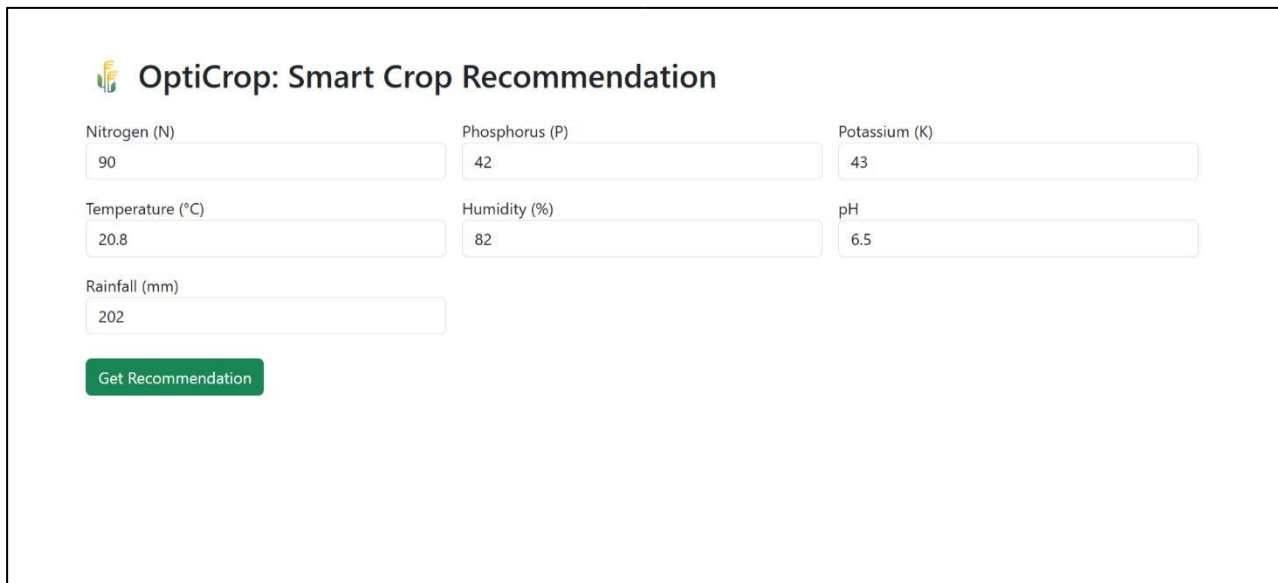
Temperature (°C) Humidity (%) pH

Rainfall (mm)

Get Recommendation

Description: OptiCrop Smart Agricultural Production Optimization Engine provides a web-based interface for intelligent crop recommendation. Users enter soil nutrient values (N, P, K), temperature, humidity, pH, and rainfall through the application. The Flask backend processes these inputs using a trained Machine Learning model to predict the most suitable crop. The system offers a simple, responsive, and user-friendly interface that helps farmers improve productivity and make datadriven agricultural decisions.

Input Section:

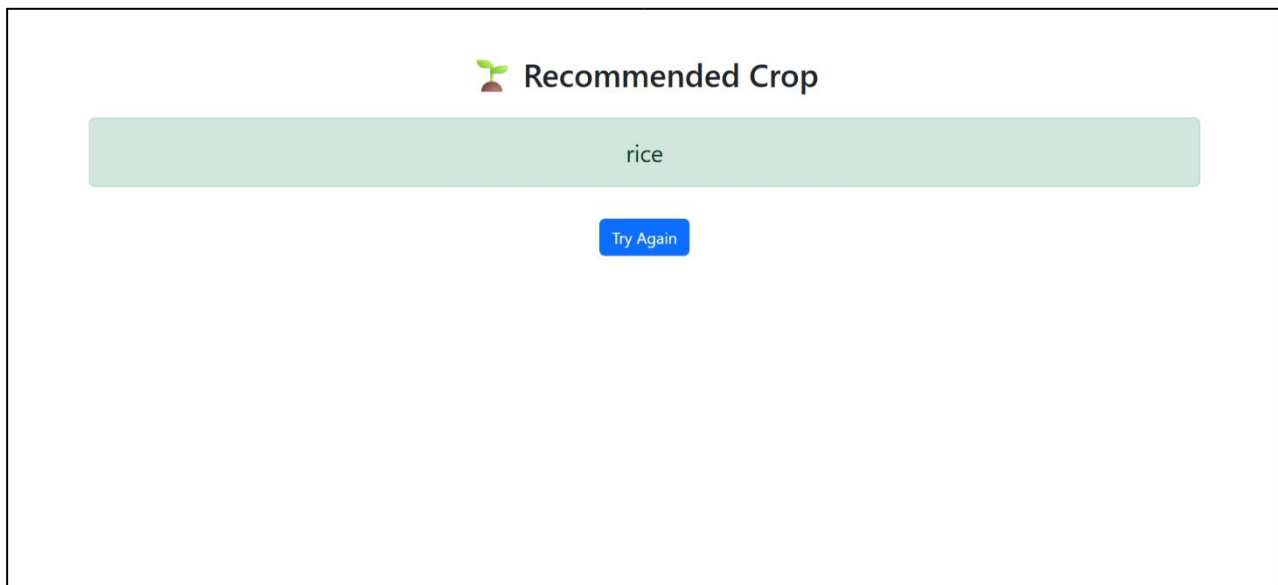


The screenshot displays the 'OptiCrop: Smart Crop Recommendation' web interface. It features a title bar with a small icon and the text 'OptiCrop: Smart Crop Recommendation'. Below the title, there are six input fields arranged in three rows. The first row contains 'Nitrogen (N)' with value '90', 'Phosphorus (P)' with value '42', and 'Potassium (K)' with value '43'. The second row contains 'Temperature (°C)' with value '20.8', 'Humidity (%)' with value '82', and 'pH' with value '6.5'. The third row contains 'Rainfall (mm)' with value '202'. At the bottom left, there is a green button labeled 'Get Recommendation'.

Parameter	Value
Nitrogen (N)	90
Phosphorus (P)	42
Potassium (K)	43
Temperature (°C)	20.8
Humidity (%)	82
pH	6.5
Rainfall (mm)	202

Description: The figure shows the sample input interface of the OptiCrop: Smart Agricultural Production Optimization Engine. Users enter soil nutrient values (Nitrogen, Phosphorus, and Potassium) along with environmental parameters such as temperature, humidity, pH, and rainfall. After clicking the "Get Recommendation" button, the system processes the inputs using a Machine Learning model and predicts the most suitable crop for cultivation. The interface is simple, userfriendly, and designed for efficient crop recommendation.

Results Section:



Description: The figure shows the result interface of the OptiCrop: Smart Agricultural Production Optimization Engine. After the user submits the required soil and environmental parameters, the Machine Learning model analyzes the input data and displays the recommended crop. In this example, the system recommends Rice as the most suitable crop for the given conditions. The interface also includes a "Try Again" button, allowing users to return to the input form and test different values. This simple and user-friendly result page helps farmers quickly identify the best crop for cultivation based on scientific analysis.

Milestone 6: Conclusion

The **OptiCrop: Smart Agricultural Production Optimization Engine** is an AI-powered smart agriculture application developed using Python, Flask, and Machine Learning technologies to support farmers in making informed agricultural decisions. The system provides intelligent crop recommendations, fertilizer suggestions, irrigation planning, and crop yield prediction based on soil and environmental conditions. By integrating machine learning models with a user-friendly web application, OptiCrop helps improve farming productivity, optimize resource utilization, and promote sustainable agricultural practices. The project demonstrates the practical application of artificial intelligence in agriculture and provides a scalable foundation for future enhancements, such as real-time weather integration, IoT sensor support, pest and disease prediction, and mobile application development.